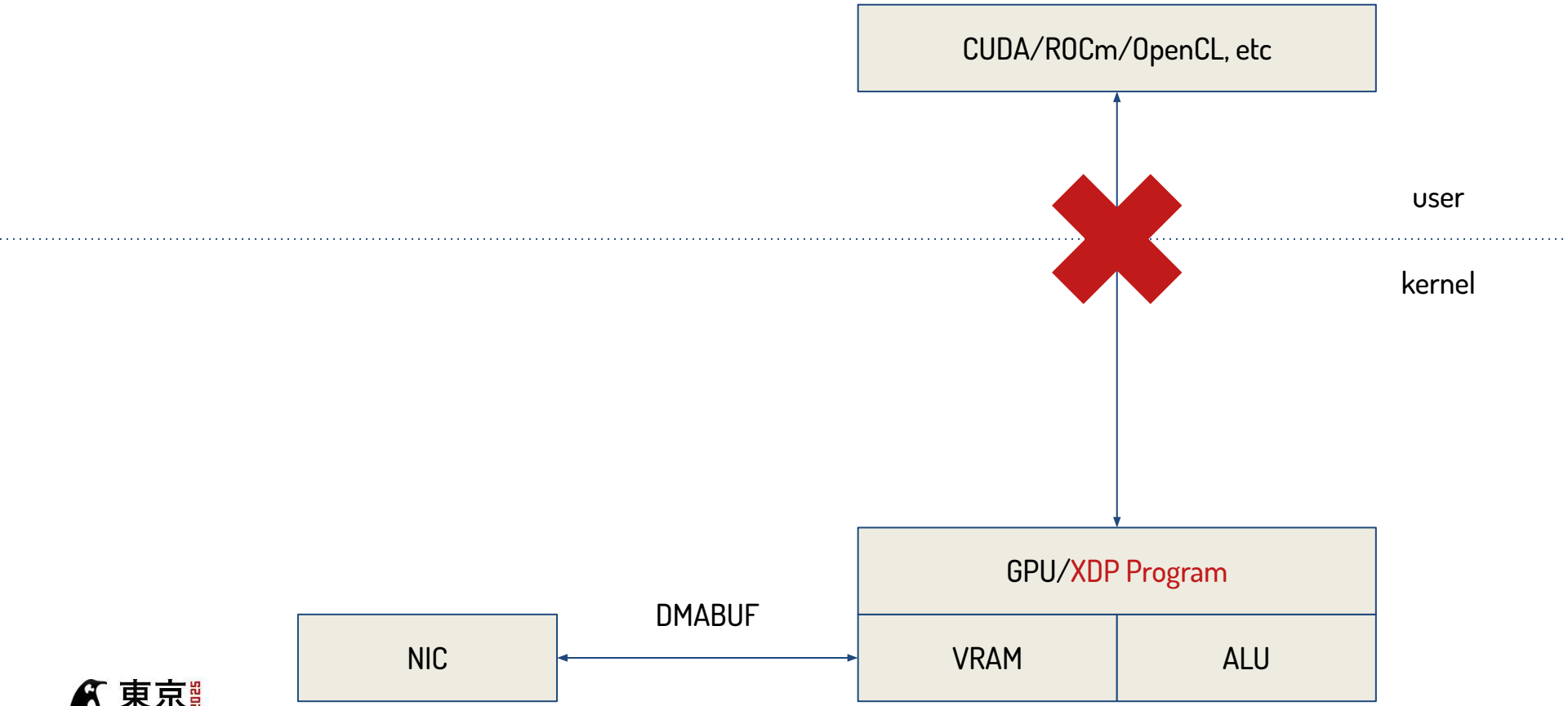
Kernel-Native Packet Processing on AMD GPUs

Taehee Yoo Rebellions



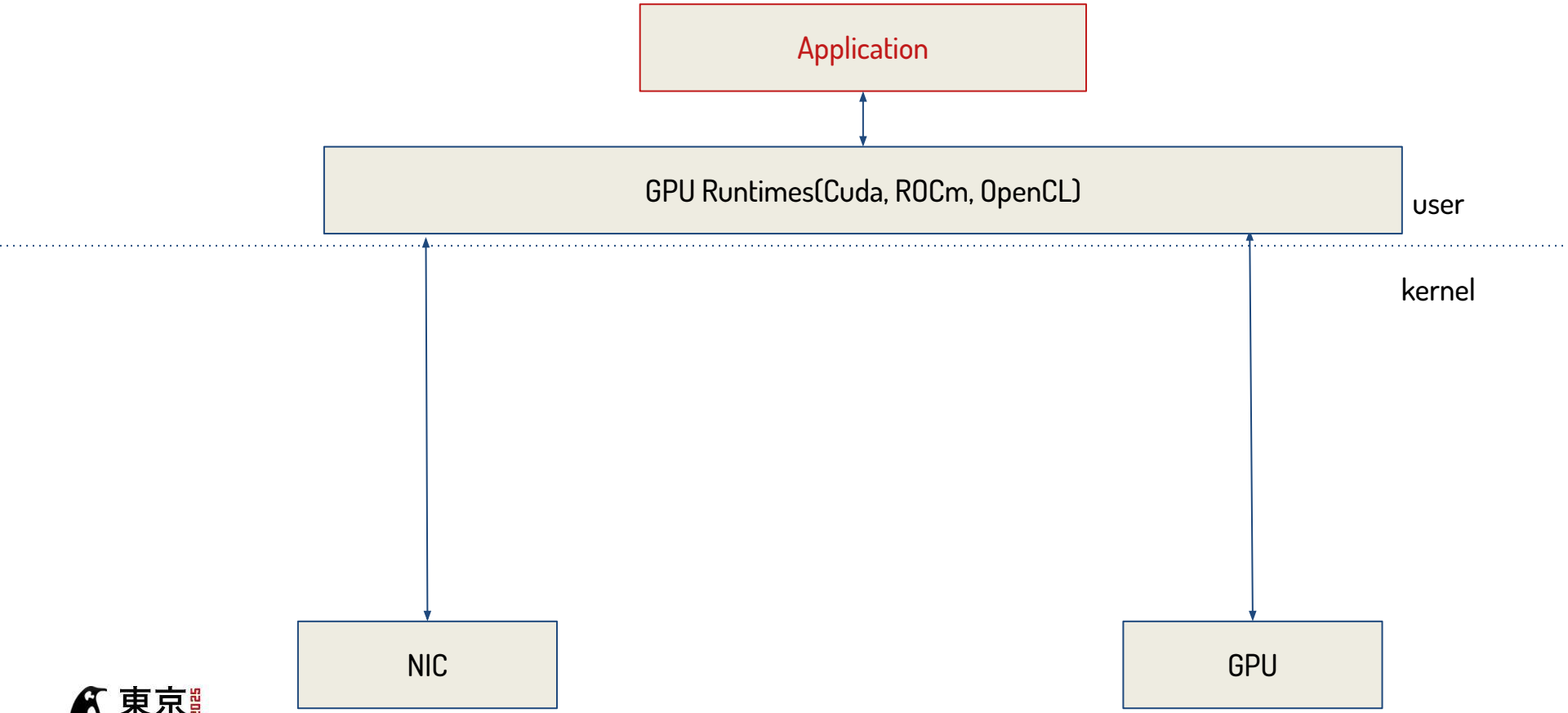
東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

The goal is to accelerate packet processing directly in the Linux kernel using GPUs, without relying on userspace runtimes.


CUDA/ROCm/OpenCL, etc
user
kernel
GPU/XDP Program
DMABUF
NIC
VRAM
ALU

Related work

- Packetshader
- APUNet
- GPU-Ether


Application
GPU Runtimes(Cuda, ROCm, OpenCL)
user
kernel
NIC
GPU

AMDGPU

Why AMDGPU

- AMDGPU is fully open.
  - Instruction set
  - Framework
    - HSA(Heterogeneous System Architecture)
    - Full open architecture, AMDGPU is working based on HSA.
  - Open Source Runtimes, UMD, and KMD
    - ROCm, HSA, amdgpu/amdkfd(kmd), etc.
- Other opened GPUs are possible.
  - Intel?

```
hostA = (float*)malloc(NUM * sizeof(float));
hostB = (float*)malloc(NUM * sizeof(float));
hostC = (float*)malloc(NUM * sizeof(float));

// initialize the input data
for (i = 0; i < NUM; i++) {
  hostB[i] = (float)i;
  hostC[i] = (float)i*100.0f;
}

HIP_ASSERT(hipMalloc((void**)&deviceA, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceB, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceC, NUM * sizeof(float)));

HIP_ASSERT(hipMemcpy(deviceB, hostB, sizeof(float), hipMemcpyHostToDevice));

HIP_ASSERT(hipMemcpy(deviceB, hostB, NUM*sizeof(float), hipMemcpyHostToDevice));
HIP_ASSERT(hipMemcpy(deviceC, hostC, NUM*sizeof(float), hipMemcpyHostToDevice));


hipLaunchKernelGGL(vectoradd_float,
                dim3(WIDTH/THREADS_PER_BLOCK_X, HEIGHT/THREADS_PER_BLOCK_Y),
                dim3(THREADS_PER_BLOCK_X, THREADS_PER_BLOCK_Y),
                0, 0,
                deviceA ,deviceB ,deviceC ,WIDTH ,HEIGHT);


HIP_ASSERT(hipMemcpy(hostA, deviceA, NUM*sizeof(float), hipMemcpyDeviceToHost));
```

```
hostA = (float*)malloc(NUM * sizeof(float));
hostB = (float*)malloc(NUM * sizeof(float));
hostC = (float*)malloc(NUM * sizeof(float));

// initialize the input data
for (i = 0; i < NUM; i++) {
  hostB[i] = (float)i;
  hostC[i] = (float)i*100.0f;
}

HIP_ASSERT(hipMalloc((void**)&deviceA, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceB, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceC, NUM * sizeof(float)));

HIP_ASSERT(hipMemcpy(deviceB, hostB, sizeof(float), hipMemcpyHostToDevice));

HIP_ASSERT(hipMemcpy(deviceB, hostB, NUM*sizeof(float), hipMemcpyHostToDevice));
HIP_ASSERT(hipMemcpy(deviceC, hostC, NUM*sizeof(float), hipMemcpyHostToDevice));


hipLaunchKernelGGL(vectoradd_float,
                dim3(WIDTH/THREADS_PER_BLOCK_X, HEIGHT/THREADS_PER_BLOCK_Y),
                dim3(THREADS_PER_BLOCK_X, THREADS_PER_BLOCK_Y),
                0, 0,
                deviceA ,deviceB ,deviceC ,WIDTH ,HEIGHT);


HIP_ASSERT(hipMemcpy(hostA, deviceA, NUM*sizeof(float), hipMemcpyDeviceToHost));
```

```
hostA = (float*)malloc(NUM * sizeof(float));
hostB = (float*)malloc(NUM * sizeof(float));
hostC = (float*)malloc(NUM * sizeof(float));

// initialize the input data
for (i = 0; i < NUM; i++) {
  hostB[i] = (float)i;
  hostC[i] = (float)i*100.0f;
}

HIP_ASSERT(hipMalloc((void**)&deviceA, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceB, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceC, NUM * sizeof(float)));

HIP_ASSERT(hipMemcpy(deviceB, hostB, sizeof(float), hipMemcpyHostToDevice));

HIP_ASSERT(hipMemcpy(deviceB, hostB, NUM*sizeof(float), hipMemcpyHostToDevice));
HIP_ASSERT(hipMemcpy(deviceC, hostC, NUM*sizeof(float), hipMemcpyHostToDevice));


hipLaunchKernelGGL(vectoradd_float,
                dim3(WIDTH/THREADS_PER_BLOCK_X, HEIGHT/THREADS_PER_BLOCK_Y),
                dim3(THREADS_PER_BLOCK_X, THREADS_PER_BLOCK_Y),
                0, 0,
                deviceA ,deviceB ,deviceC ,WIDTH ,HEIGHT);


HIP_ASSERT(hipMemcpy(hostA, deviceA, NUM*sizeof(float), hipMemcpyDeviceToHost));
```

```
hostA = (float*)malloc(NUM * sizeof(float));
hostB = (float*)malloc(NUM * sizeof(float));
hostC = (float*)malloc(NUM * sizeof(float));

// initialize the input data
for (i = 0; i < NUM; i++) {
  hostB[i] = (float)i;
  hostC[i] = (float)i*100.0f;
}

HIP_ASSERT(hipMalloc((void**)&deviceA, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceB, NUM * sizeof(float)));
HIP_ASSERT(hipMalloc((void**)&deviceC, NUM * sizeof(float)));

HIP_ASSERT(hipMemcpy(deviceB, hostB, sizeof(float), hipMemcpyHostToDevice));

HIP_ASSERT(hipMemcpy(deviceB, hostB, NUM*sizeof(float), hipMemcpyHostToDevice));
HIP_ASSERT(hipMemcpy(deviceC, hostC, NUM*sizeof(float), hipMemcpyHostToDevice));

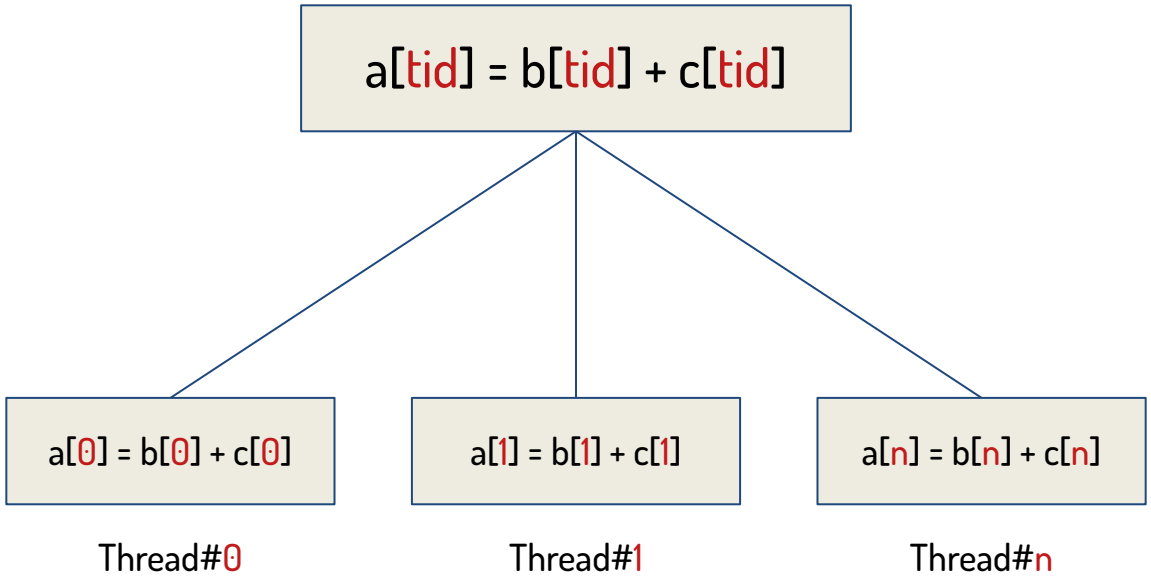

hipLaunchKernelGGL(vectoradd_float,
                   dim3(WIDTH/THREADS_PER_BLOCK_X, HEIGHT/THREADS_PER_BLOCK_Y),
                   dim3(THREADS_PER_BLOCK_X, THREADS_PER_BLOCK_Y),
                   0, 0,
                   deviceA ,deviceB ,deviceC ,WIDTH ,HEIGHT);


HIP_ASSERT(hipMemcpy(hostA, deviceA, NUM*sizeof(float), hipMemcpyDeviceToHost));
```

```
__global__ void  vectoradd_float(float* __restrict__ a,
                                 const float* __restrict__ b,
                                 const float* __restrict__ c,
                                 int width,
                                 int height)

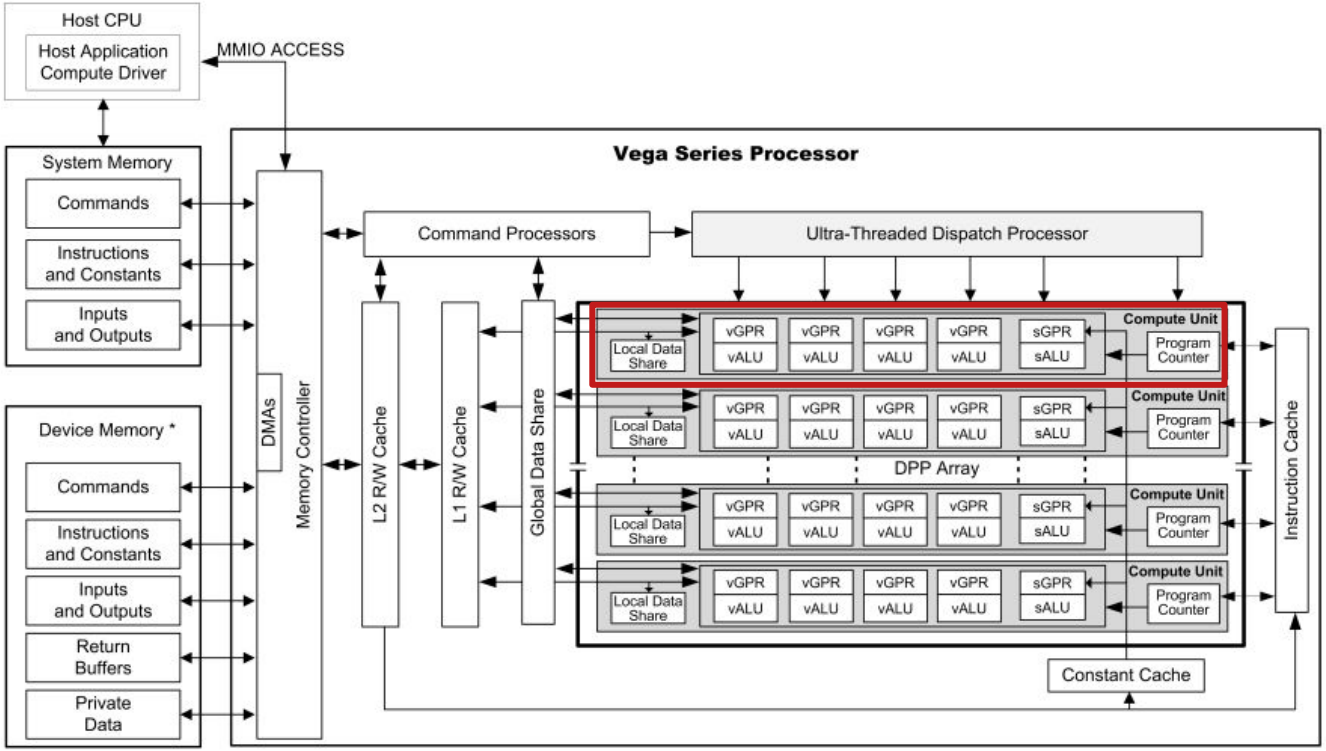
{
        int x = hipBlockDim_x * hipBlockIdx_x + hipThreadIdx_x;
        int y = hipBlockDim_y * hipBlockIdx_y + hipThreadIdx_y;
        int i = y * width + x;

        if ( i < (width * height))
                a[i] = b[i] + c[i];
}
```


a[tid] = b[tid] + c[tid]
a[0] = b[0] + c[0]
a[1] = b[1] + c[1]
a[n] = b[n] + c[n]
Thread#0
Thread#1
Thread#n

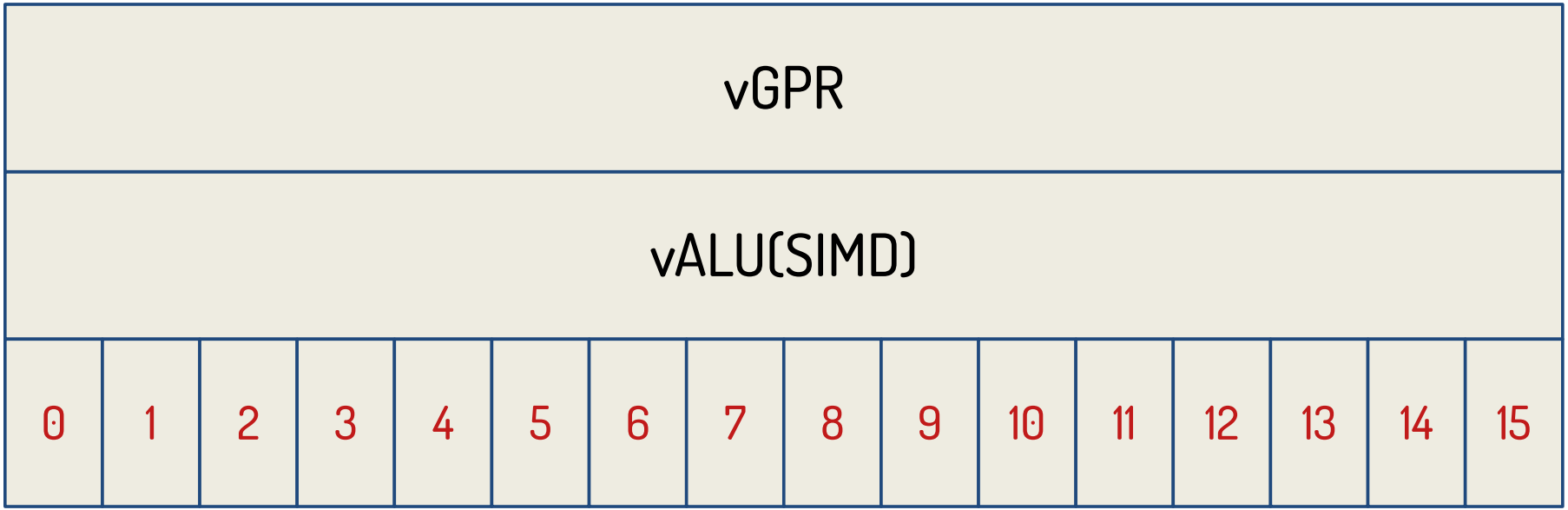
Glossaries

- Dispatch
  - Launch/execute GPU Kernel
- Workitem
  - Each thread
- Workgroup
  - Group of Workitem(Thread)
- Scalar
  - Operation of control flow
- Vector
  - Operation of normal like arithmetic, etc.

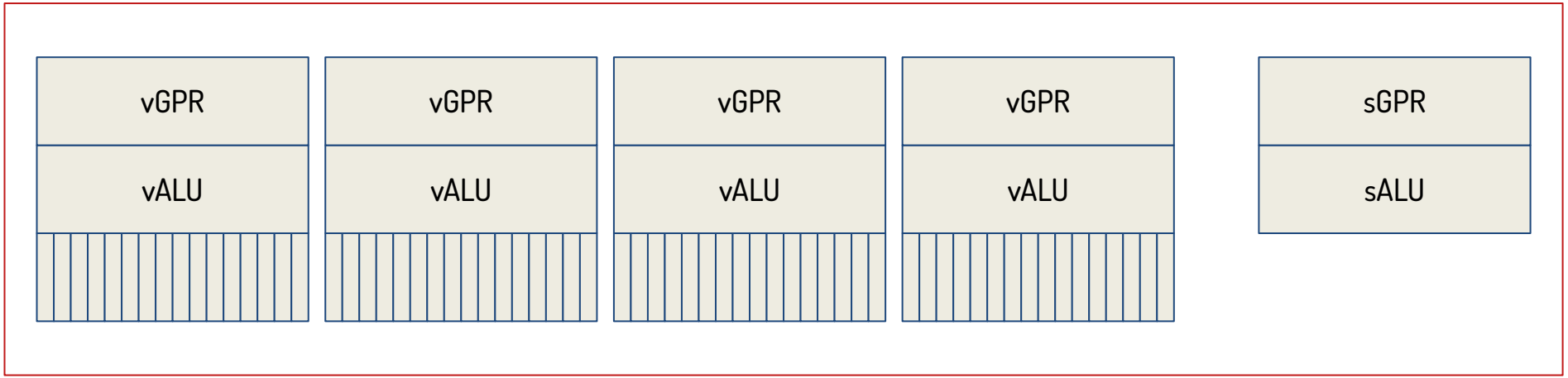


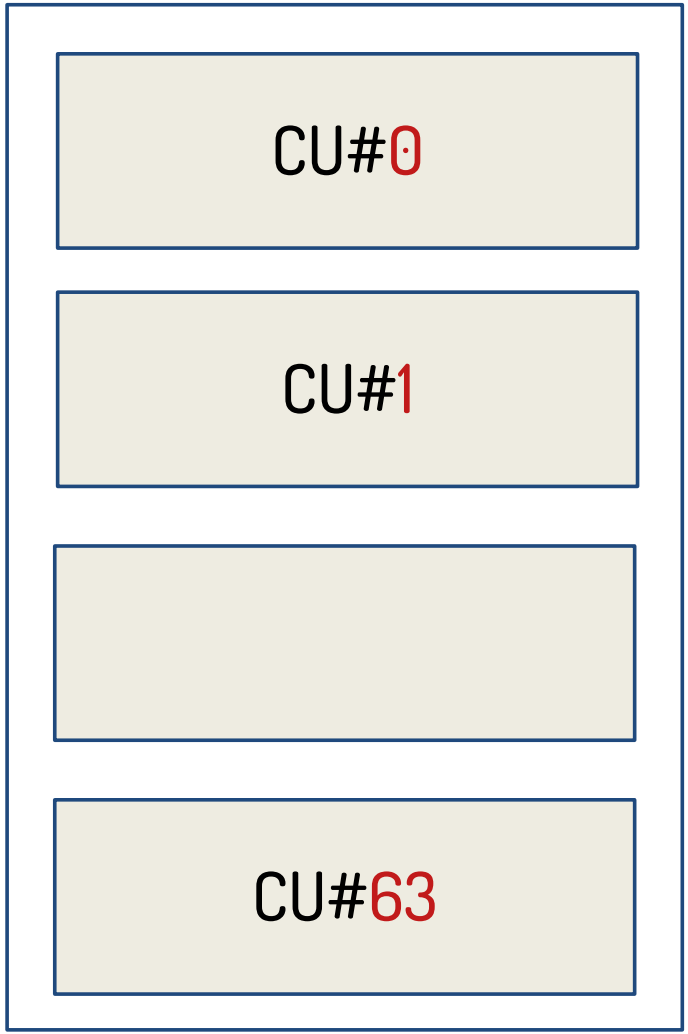
*Discrete GPU – Physical Device Memory; APU – Region of system for GPU direct access

Figure 1. AMD GCN VEGA Generation Series Block Diagram


vGPR
vALU(SIMD)
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

CU#0




CU#0
CU#1
CU#63

Register

- Each register is 32bits.
- Each Thread can have up to 256 vGPRs
  - CDNA(Computing for Enterprise) can have 512 vGPRs
  - Register Allocation

Stack Memory

- Scratch pad
  - VRAM for thread-private area.
  - No need to calculate correct address, just use it.
- Register as Stack Memory
  - 256 vGPRs can store 1KB.
  - BPF requires ~512B as a stack memory.

Instructions

- Most instructions support 64bits.
  - Multiply doesn't support 64bits.
  - v_mul_lo_u32 //mul for low 32bits
  - v_mul_hi_h32 // mul for high 32bits.
- AMDGPU supports all necessary BPF instructions except divide and mod.
  - Similar to Netronome and some RISC machine.
- BPF supports only integer, but GPU supports floating type.
  - Will BPF support floating type?
- Instructions should use gpu virtual address, not hardware address or offset.

GPU Memory

- VRAM
  - on chip device memory
  - fast
- GTT
  - Host memory
  - slow for dGPU
  - iGPU doesn't have VRAM, use GTT only.

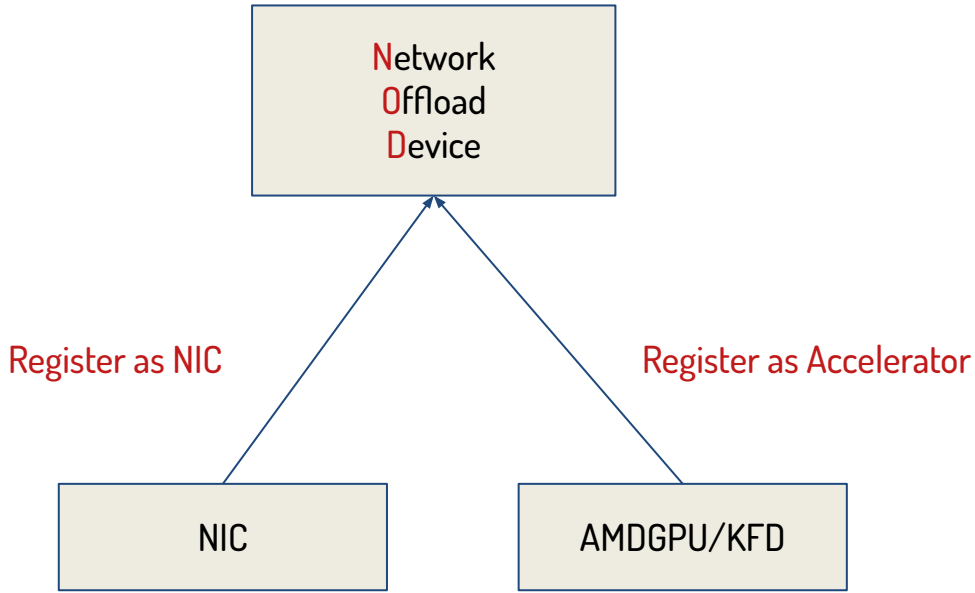
GPU Kernel Dispatch

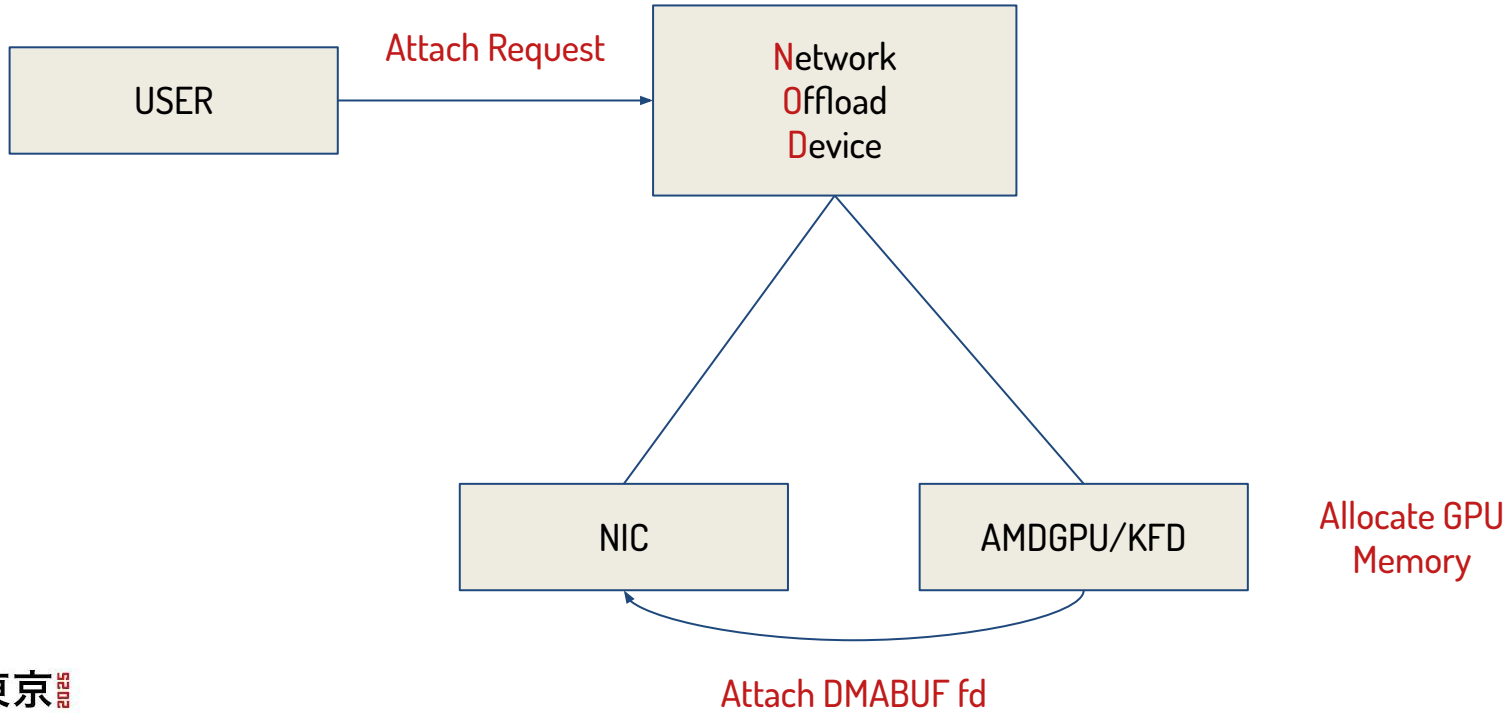
- GPU Kernel is dispatched through AQL(Architected Queueing Language).
  - AQL is defined by HSA.
  - Build AQL packet.
    - Build AQL header
    - Insert AMDGPU bytecode to AQL

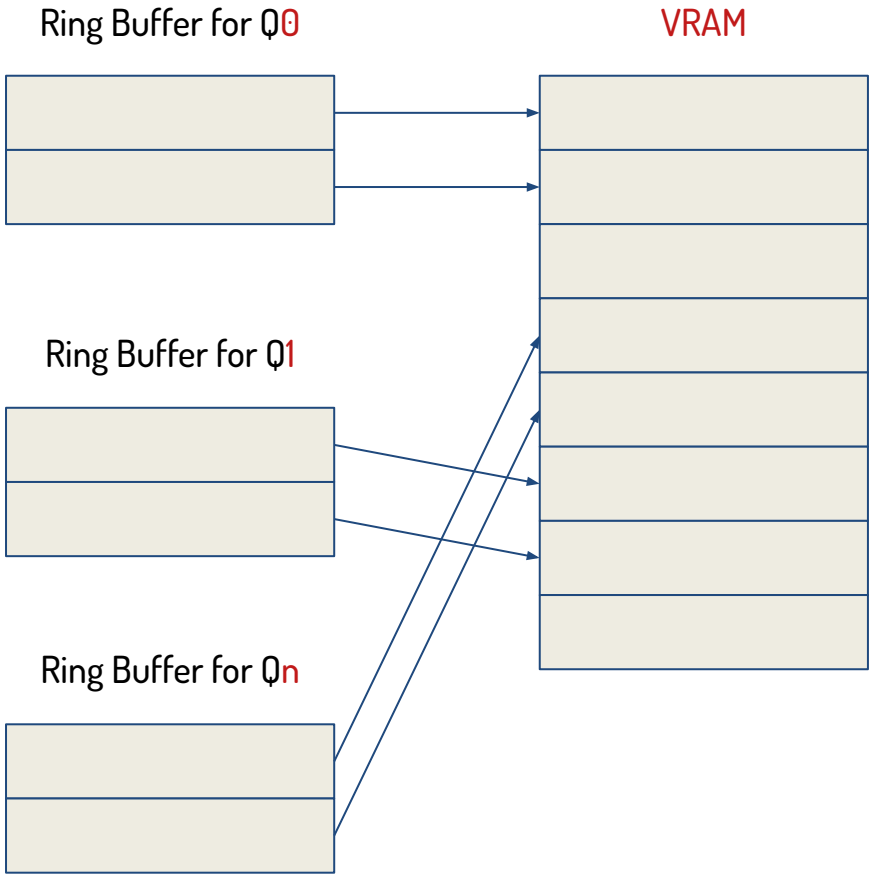
Table 2–7 Architected Queuing Language (AQL) kernel dispatch packet format

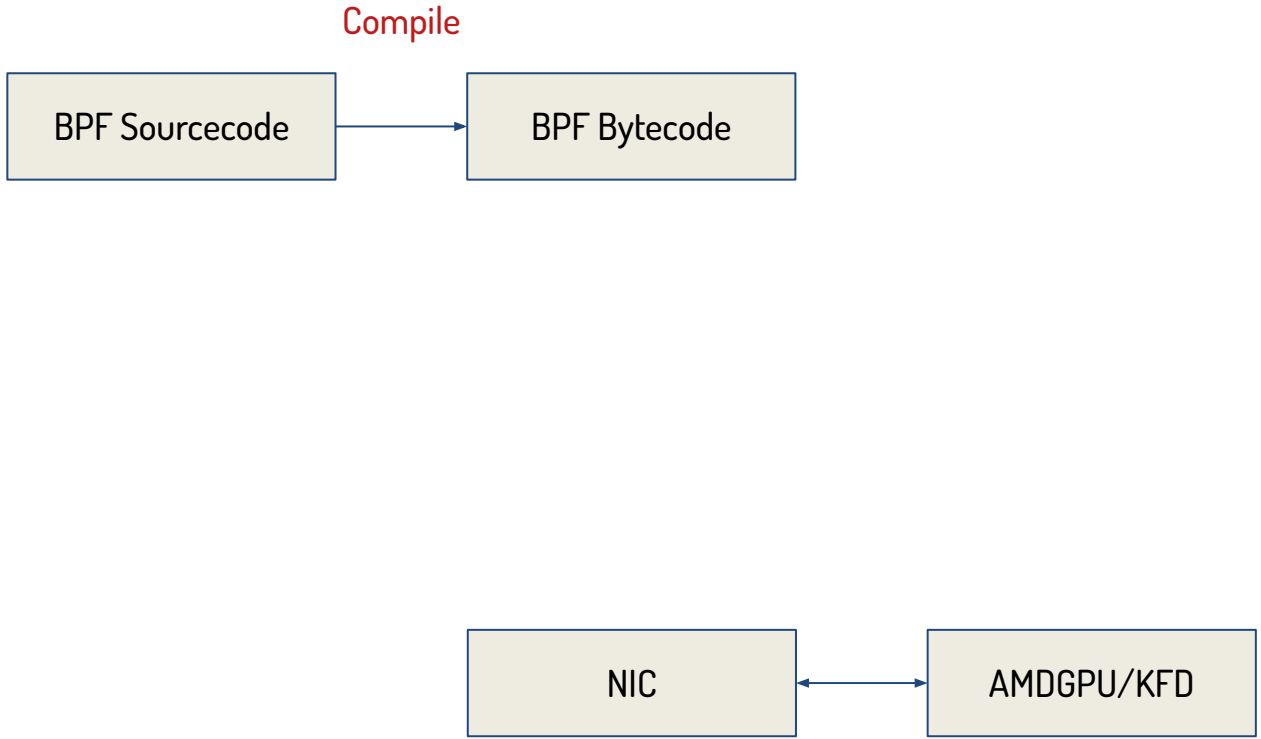
Bits	Field Name	Description
15:0	header	Packet header, see 2.9.1 Packet header (on page 25).
17:16	dimensions	Number of dimensions specified in gridSize. Valid values are 1, 2, or 3.
31:18		Reserved, must be 0.
47:32	workgroup_size_x	x dimension of work-group (measured in work-items).
63:48	workgroup_size_y	y dimension of work-group (measured in work-items).
79:64	workgroup_size_z	z dimension of work-group (measured in work-items).
95:80		Reserved, must be 0.
127:96	grid_size_x	x dimension of grid (measured in work-items).
159:128	grid_size_y	y dimension of grid (measured in work-items).
191:160	grid_size_z	z dimension of grid (measured in work-items).
223:192	private_segment_size_bytes	Total size in bytes of private memory allocation request (per work-item).
255:224	group_segment_size_bytes	Total size in bytes of group memory allocation request (per work-group).
319:256	kernel_object	Handle for an object in memory that includes an implementation-defined executable ISA image for the kernel.
383:320	kernarg_address	Address of memory containing kernel arguments.
447:384		Reserved, must be 0.
511:448	completion_signal	HSA signaling object handle used to indicate completion of the job.

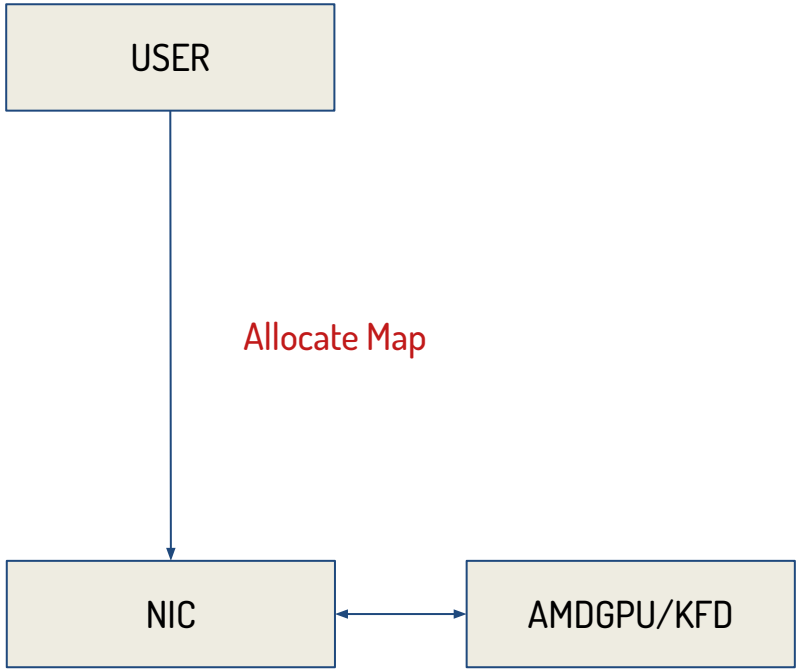
Control Plane

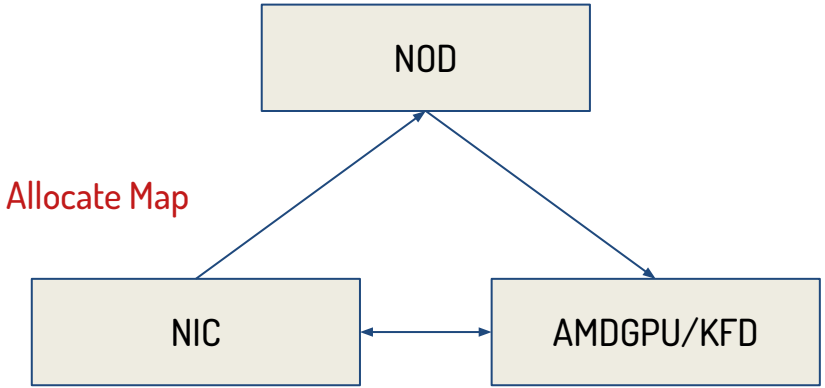

Network
Offload
Device
Register as NIC
Register as Accelerator
NIC
AMDGPU/KFD

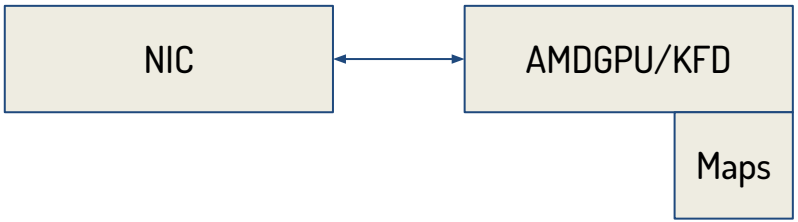

USER
Attach Request
Network
Offload
Device
NIC
AMDGPU/KFD
Allocate GPU
Memory
Attach DMABUF fd


Ring Buffer for Q0
VRAM
Ring Buffer for Q1
Ring Buffer for Qn

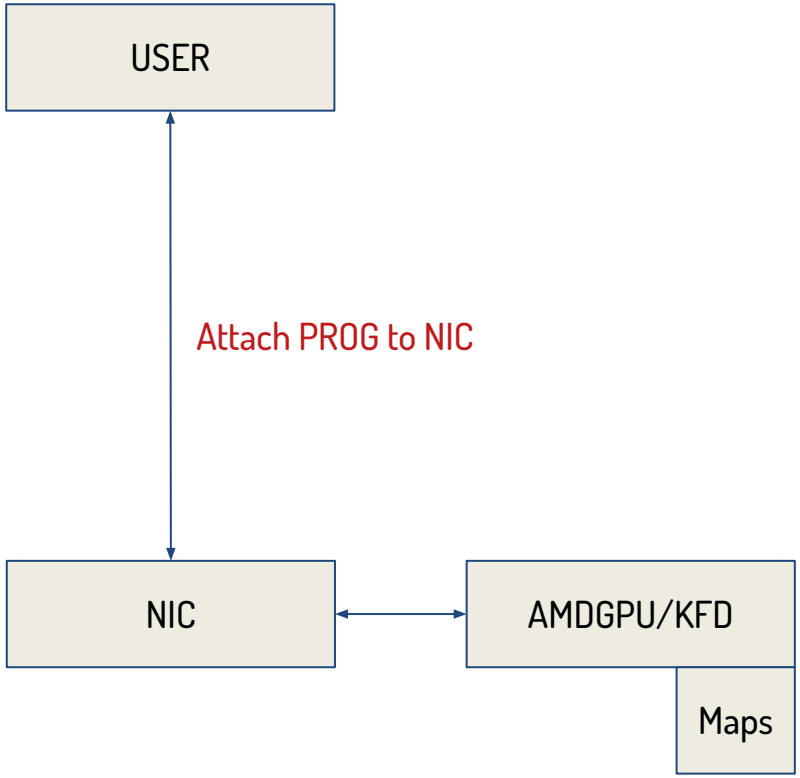

Compile
BPF Sourcecode
BPF Bytecode
NIC
AMDGPU/KFD

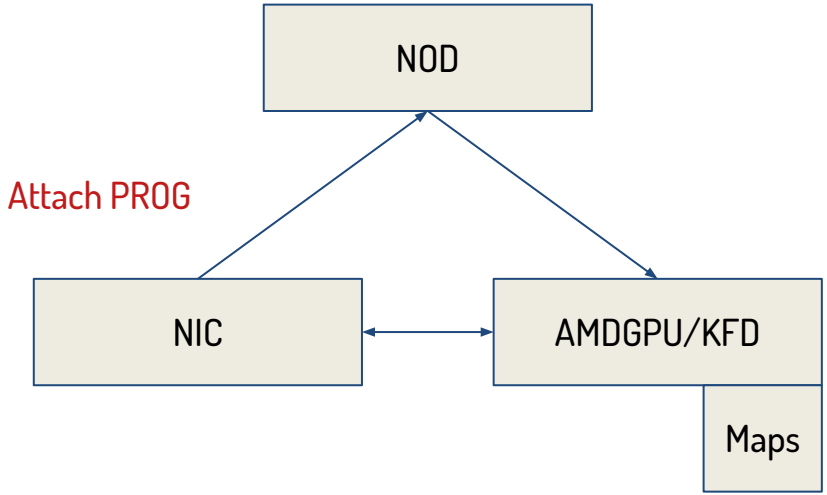

USER
Allocate Map
NIC
AMDGPU/KFD

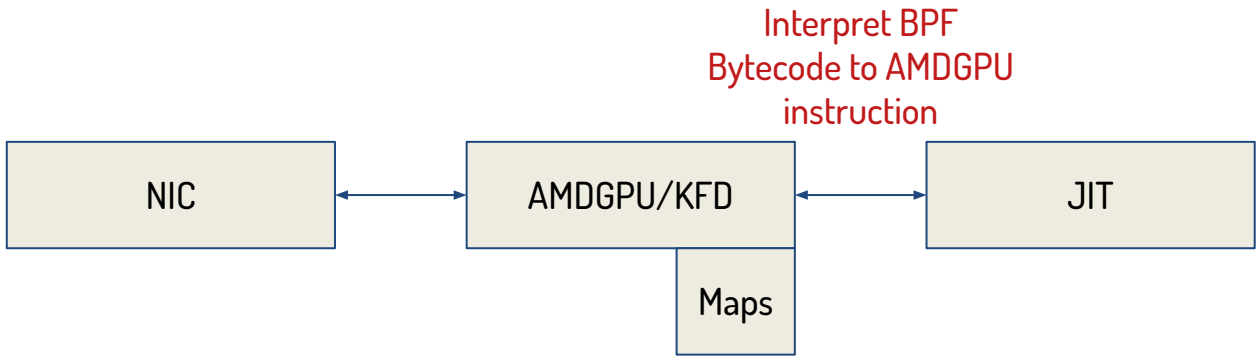

NOD
Allocate Map
NIC
AMDGPU/KFD

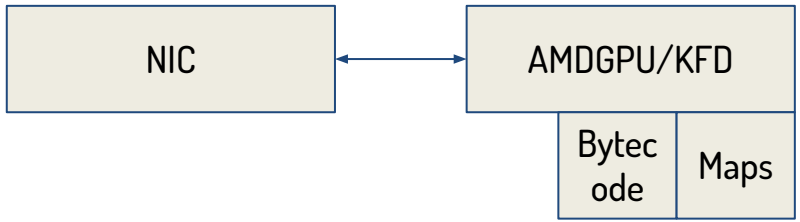


Allocate VRAM/GTT
for Maps

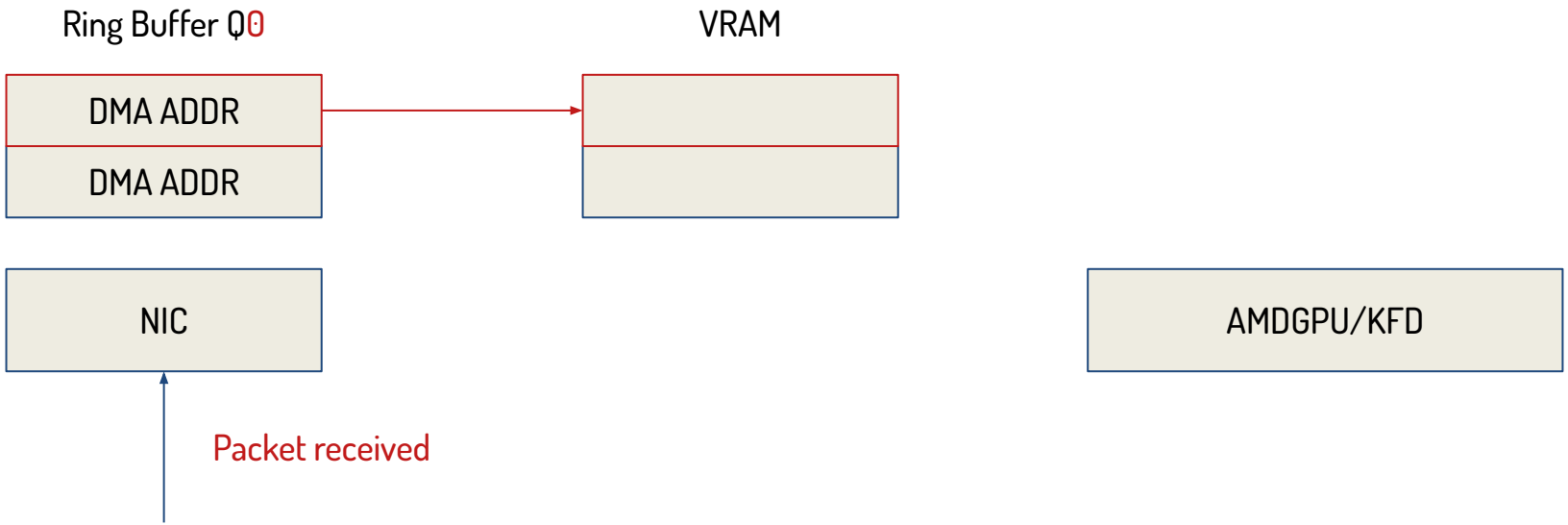

USER
Attach PROG to NIC
NIC
AMDGPU/KFD
Maps

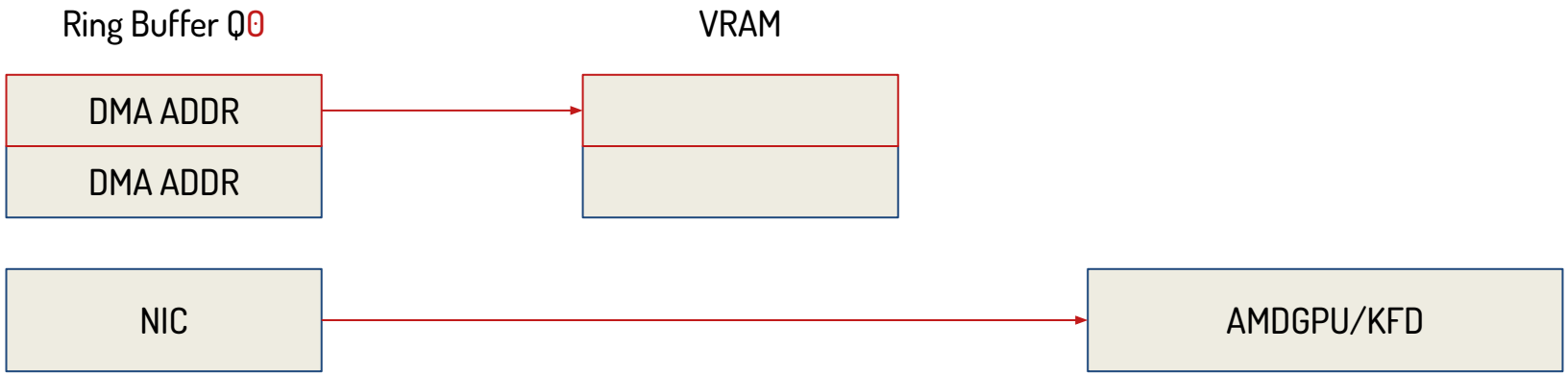

NOD
Attach PROG
NIC
AMDGPU/KFD
Maps

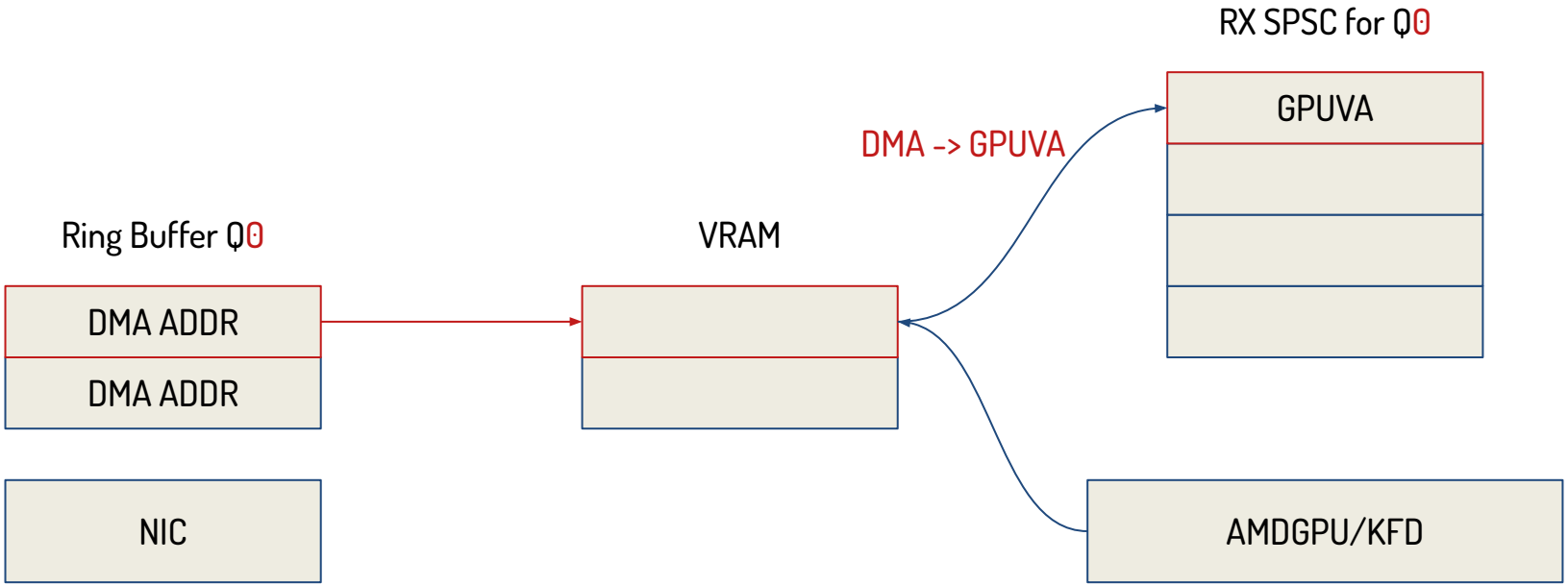

Interpret BPF
Bytecode to AMDGPU
instruction
NIC
AMDGPU/KFD
JIT
Maps

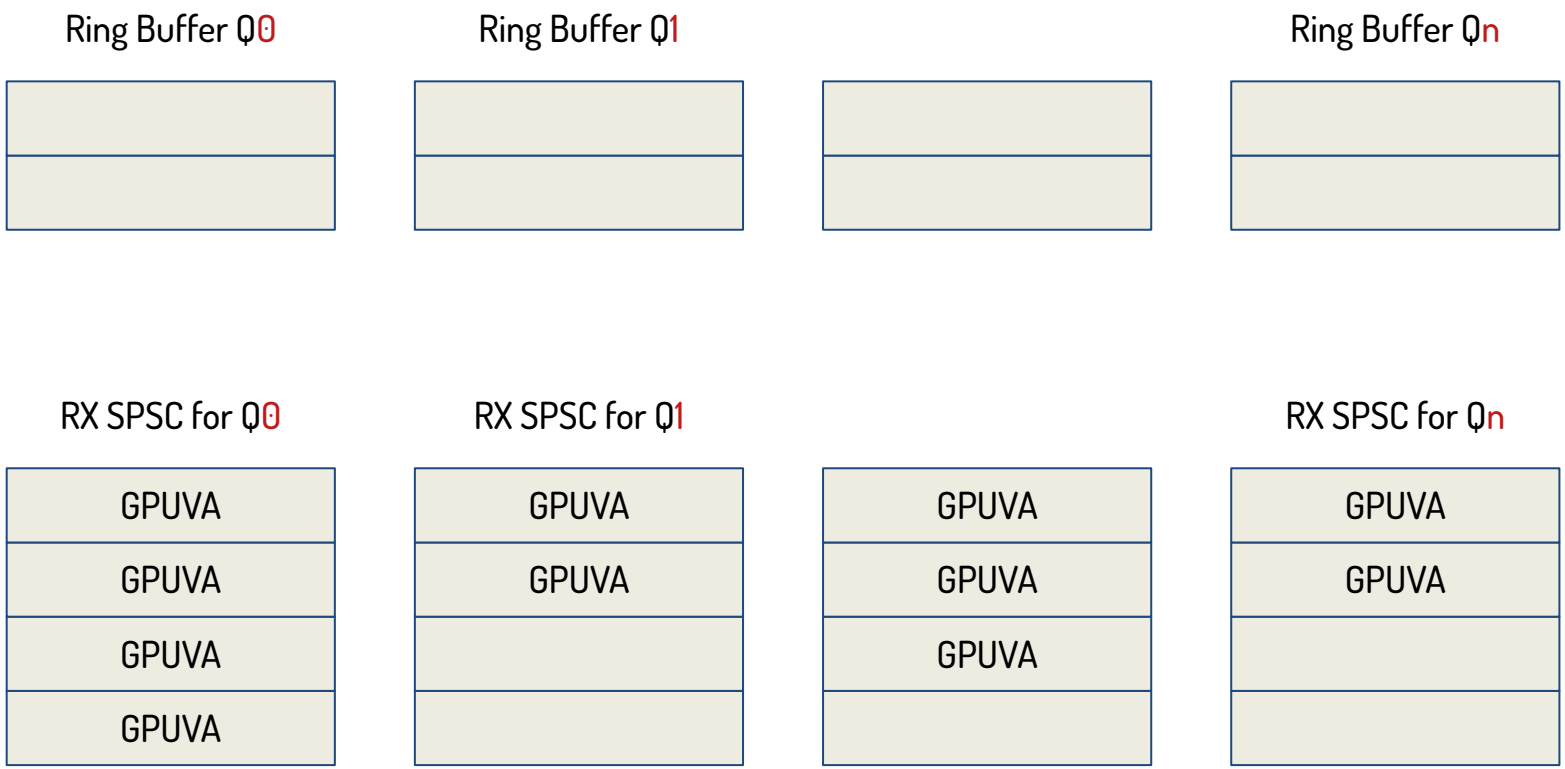

NIC
AMDGPU/KFD
Bytec
ode
Maps

Data Plane


Ring Buffer Q0
VRAM
DMA ADDR
DMA ADDR
NIC
AMDGPU/KFD
Packet received


Ring Buffer Q0
VRAM
DMA ADDR
DMA ADDR
NIC
AMDGPU/KFD


RX SPSC for Q0
GPUVA
DMA -> GPUVA
Ring Buffer Q0
VRAM
DMA ADDR
DMA ADDR
NIC
AMDGPU/KFD


Ring Buffer Q0
Ring Buffer Q1
Ring Buffer Qn
RX SPSC for Q0
RX SPSC for Q1
RX SPSC for Qn
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA

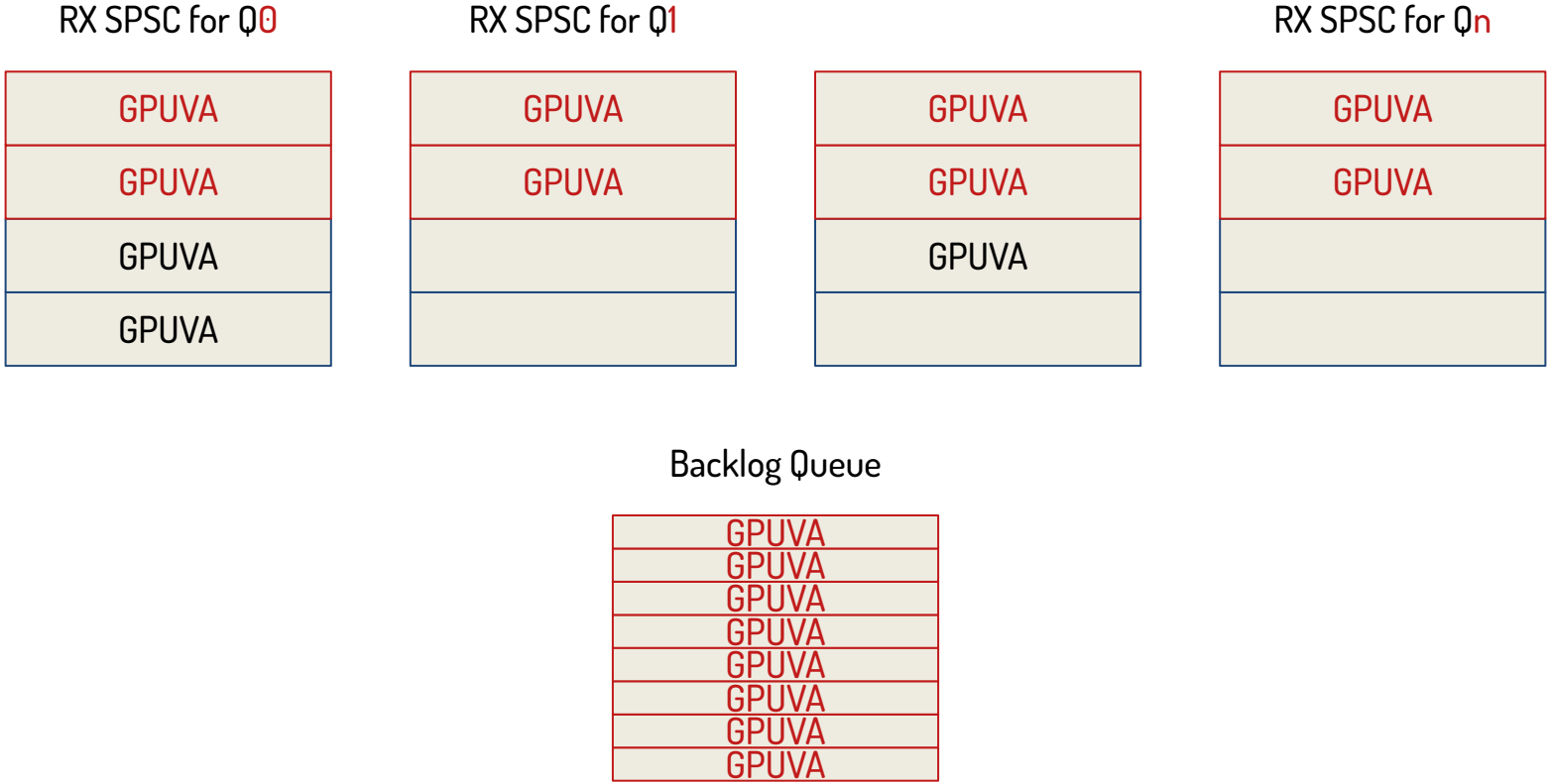

RX SPSC for Q0
GPUVA
GPUVA
GPUVA
GPUVA
RX SPSC for Q1
GPUVA
GPUVA
RX SPSC for Qn
GPUVA
GPUVA
Backlog Queue
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA
GPUVA

Table 2–7 Architected Queuing Language (AQL) kernel dispatch packet format

Bits	Field Name	Description
15:0	header	Packet header, see 2.9.1 Packet header (on page 25).
17:16	dimensions	Number of dimensions specified in gridSize. Valid values are 1, 2, or 3.
31:18		Reserved, must be 0.
47:32	workgroup_size_x	x dimension of work-group (measured in work-items).
63:48	workgroup_size_y	y dimension of work-group (measured in work-items).
79:64	workgroup_size_z	z dimension of work-group (measured in work-items).
95:80		Reserved, must be 0.
127:96	grid_size_x	x dimension of grid (measured in work-items).
159:128	grid_size_y	y dimension of grid (measured in work-items).
191:160	grid_size_z	z dimension of grid (measured in work-items).
223:192	private_segment_size_bytes	Total size in bytes of private memory allocation request (per work-item).
255:224	group_segment_size_bytes	Total size in bytes of group memory allocation request (per work-group).
319:256	kernel_object	Handle for an object in memory that includes an implementation-defined executable ISA image for the kernel.
383:320	kernarg_address	Address of memory containing kernel arguments.
447:384		Reserved, must be 0.
511:448	completion_signal	HSA signaling object handle used to indicate completion of the job.

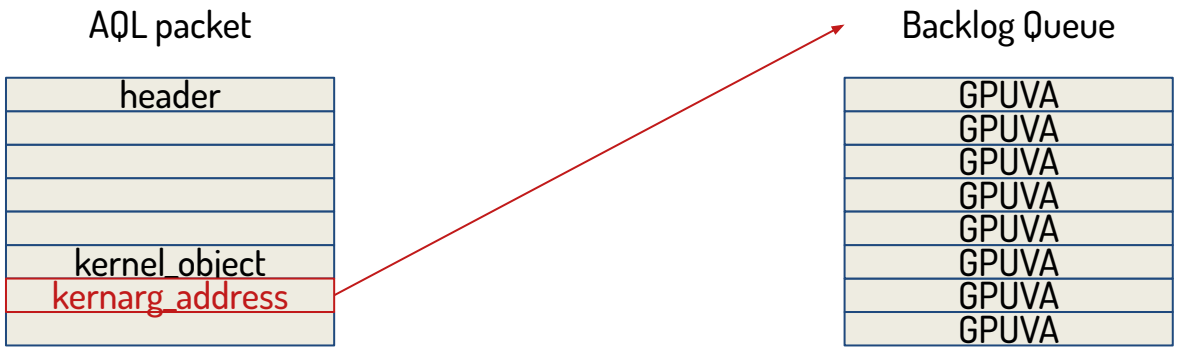
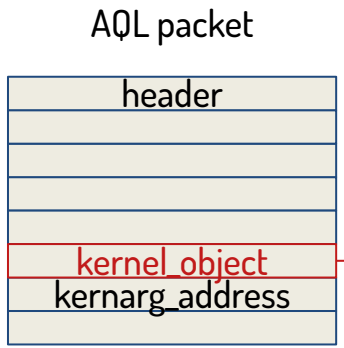


Table 2–7 Architected Queuing Language (AQL) kernel dispatch packet format

Bits	Field Name	Description
15:0	header	Packet header, see 2.9.1 Packet header (on page 25).
17:16	dimensions	Number of dimensions specified in gridSize. Valid values are 1, 2, or 3.
31:18		Reserved, must be 0.
47:32	workgroup_size_x	x dimension of work-group (measured in work-items).
63:48	workgroup_size_y	y dimension of work-group (measured in work-items).
79:64	workgroup_size_z	z dimension of work-group (measured in work-items).
95:80		Reserved, must be 0.
127:96	grid_size_x	x dimension of grid (measured in work-items).
159:128	grid_size_y	y dimension of grid (measured in work-items).
191:160	grid_size_z	z dimension of grid (measured in work-items).
223:192	private_segment_size_bytes	Total size in bytes of private memory allocation request (per work-item).
255:224	group_segment_size_bytes	Total size in bytes of group memory allocation request (per work-group).
319:256	kernel_object	Handle for an object in memory that includes an implementation-defined executable ISA image for the kernel.
383:320	kernarg_address	Address of memory containing kernel arguments.
447:384		Reserved, must be 0.
511:448	completion_signal	HSA signaling object handle used to indicate completion of the job.



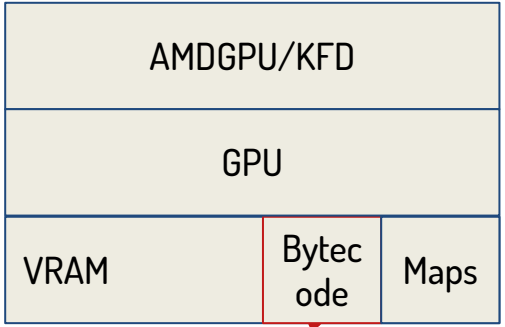
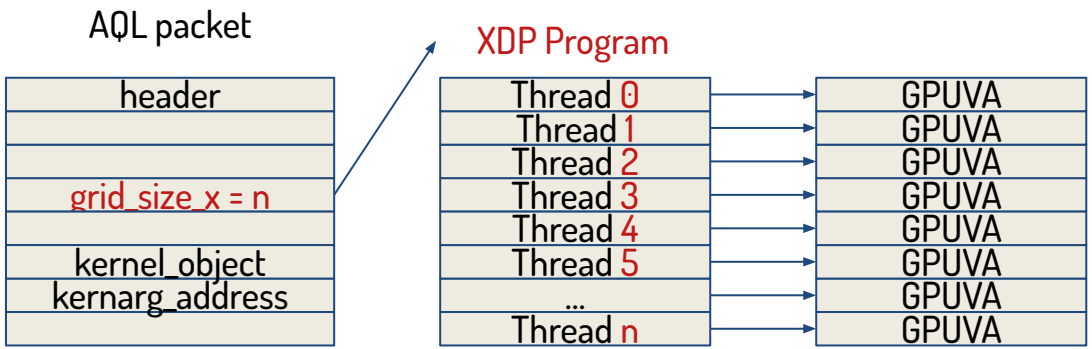
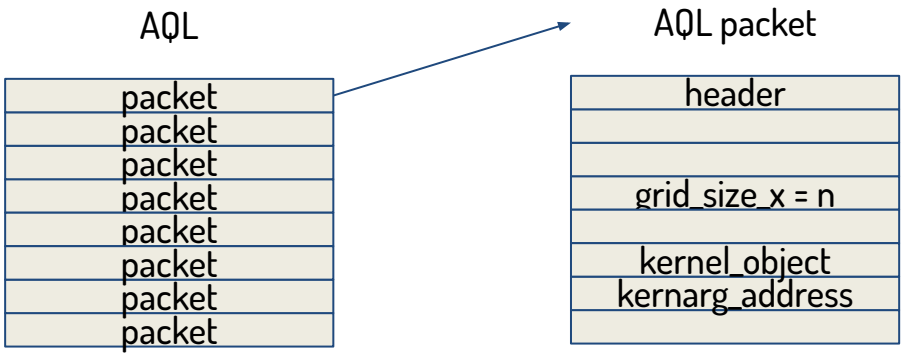
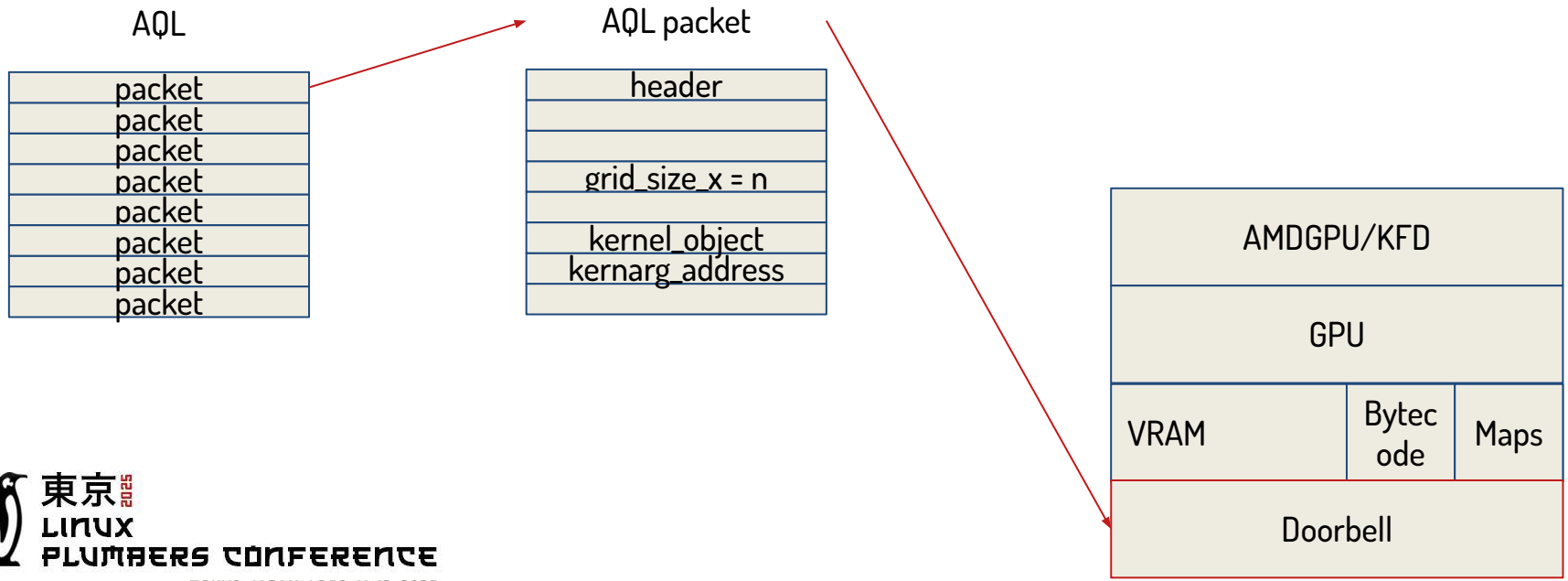


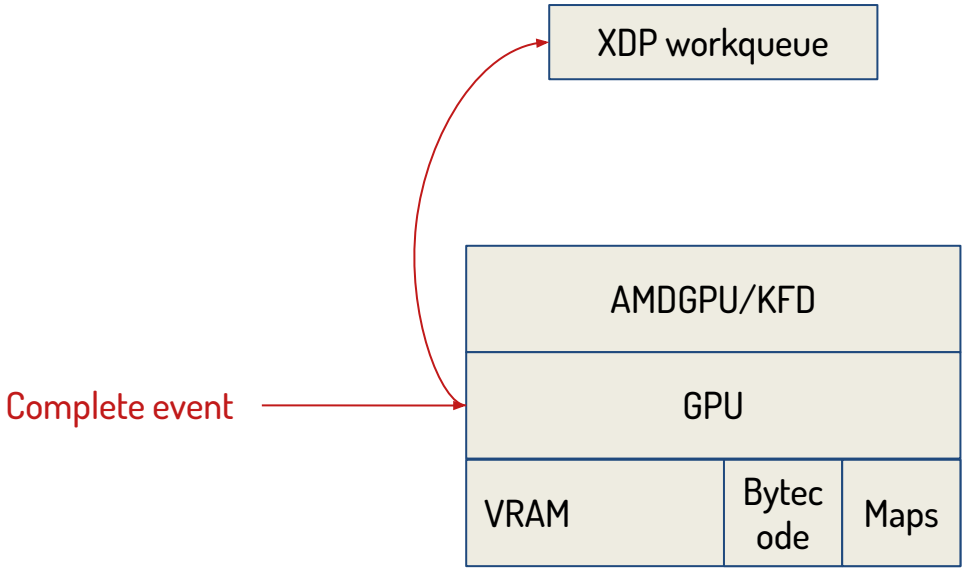
Table 2–7 Architected Queuing Language (AQL) kernel dispatch packet format

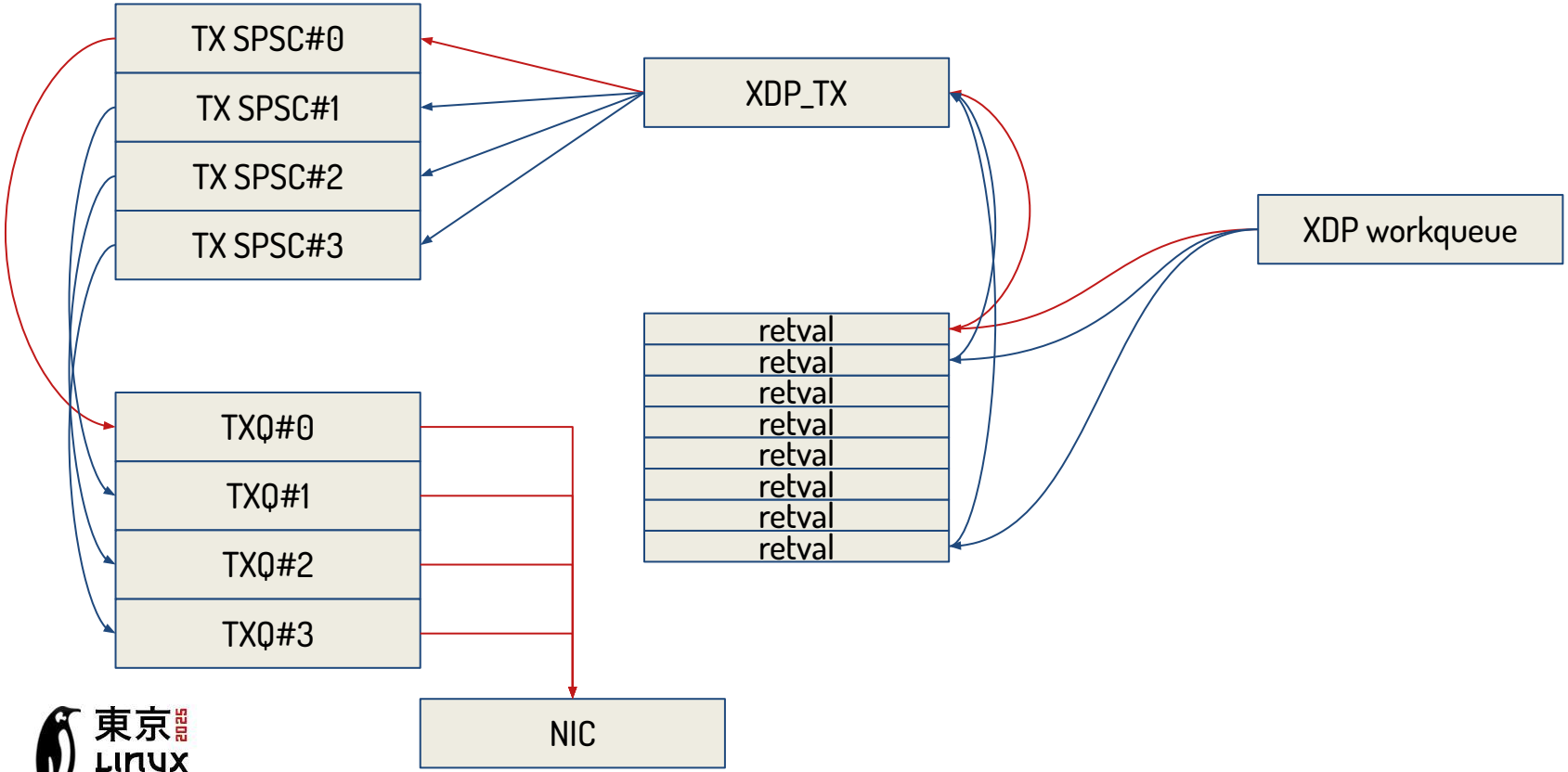
Bits	Field Name	Description
15:0	header	Packet header, see 2.9.1 Packet header (on page 25).
17:16	dimensions	Number of dimensions specified in gridSize. Valid values are 1, 2, or 3.
31:18		Reserved, must be 0.
47:32	workgroup_size_x	x dimension of work-group (measured in work-items).
63:48	workgroup_size_y	y dimension of work-group (measured in work-items).
79:64	workgroup_size_z	z dimension of work-group (measured in work-items).
95:80		Reserved, must be 0.
127:96	grid_size_x	x dimension of grid (measured in work-items).
159:128	grid_size_y	y dimension of grid (measured in work-items).
191:160	grid_size_z	z dimension of grid (measured in work-items).
223:192	private_segment_size_bytes	Total size in bytes of private memory allocation request (per work-item).
255:224	group_segment_size_bytes	Total size in bytes of group memory allocation request (per work-group).
319:256	kernel_object	Handle for an object in memory that includes an implementation-defined executable ISA image for the kernel.
383:320	kernarg_address	Address of memory containing kernel arguments.
447:384		Reserved, must be 0.
511:448	completion_signal	HSA signaling object handle used to indicate completion of the job.




AQL
packet
packet
packet
packet
packet
packet
packet
packet
AQL packet
header
grid_size_x = n
kernel_object
kernarg_address


AQL
packet
packet
packet
packet
packet
packet
packet
packet
AQL packet
header
grid_size_x = n
kernel_object
kernarg_address
AMDGPU/KFD
GPU
VRAM
Bytec
ode
Maps
Doorbell


XDP workqueue
AMDGPU/KFD
Complete event
GPU
VRAM
Bytec
ode
Maps


TX SPSC#0
TX SPSC#1
TX SPSC#2
TX SPSC#3
XDP_TX
XDP workqueue
retval
retval
retval
retval
retval
retval
retval
retval
TXQ#0
TXQ#1
TXQ#2
TXQ#3
NIC

Issues/TODO

Issues

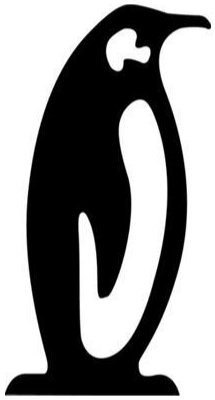
- Redesign
  - Control plane is PoC design, redesign is needed.
- Optimize XDP_PASS
  - VRAM -> Host memory copy with GPU side DMA
- Maps
  - More maps
  - Convert per-thread internally?
- Performance
  - 12Mpps bottleneck because of Ring Buffer Full
  - ringbuffer overhead

TODO

- Create for-kernel APIs of amdgpu/kfd
- support kernel dispatch API in the KMD
  - currently userspace runtime.
- Implement more feature
  - Crypto, Netfilter, TC, etc,
- Optimize Dataplane
  - redesign needed?

TODO

- Support more AMDGPUs
  - Currently only GCN is supported.
- Support other vendors
  - Intel, PowerVR?
- Support more NICs
  - Currently only Broadcom Thor+



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

